



به نام خدا
دانشگاه تهران
دانشکده مهندسی برق و کامپیوتر



مهسا ابرغانی اقدام - ۸۱۰۱۰۳۰۵۳ - تمرین چهارم درس سامانه های یادگیری ماشین توزیع شده

۱:

وارد کردن کتابخانه های مورد نیاز

```
#1- الف
import time
start_time = time.time()
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset
from accelerate import Accelerator
```

داده های آموزش load شده و سپس به نوع داده ی float32 تبدیل شده و در آخر با استفاده از TensorDataset به دیتاست PyTorch تبدیل می شوند.

```
accelerator = Accelerator()
test_x = np.load('./test_data/test_x.npy').astype(np.float32)
test_y = np.load('./test_data/test_y.npy').astype(np.float32)
test_dataset = TensorDataset(torch.tensor(test_x).float(),
                               torch.tensor(test_y).long())

train_x = np.load("./train_data/train_x.npy").astype(np.float32)
train_y = np.load("./train_data/train_y.npy").astype(np.float32)
train_dataset = TensorDataset(torch.tensor(train_x).float(),
                               torch.tensor(train_y).long())
```

```
batch_size = 100 #تعداد نمونه ها
n_iters = 3000 #تعداد کل تکرار ها
num_epochs = n_iters / (len(train_dataset) / batch_size)
#تعداد دوره ها بر اساس تکرار ها و اندازه داده ها
num_epochs = int(num_epochs)
```

داده‌های آموزشی به صورت تصادفی (shuffle=True) و داده‌های آزمایشی بدون جابجایی (shuffle=False) بارگذاری می‌شوند.

Load کردن دیتا :

```
train_loader = torch.utils.data.DataLoader(dataset=train_dataset,
                                             batch_size=batch_size,
                                             shuffle=True)

test_loader = torch.utils.data.DataLoader(dataset=test_dataset,
                                           batch_size=batch_size,
                                           shuffle=False)
```

شبکه ی عصبی:

```
class FeedforwardNeuralNetModel(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super(FeedforwardNeuralNetModel, self).__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.bn1 = nn.BatchNorm1d(hidden_dim)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.bn2 = nn.BatchNorm1d(hidden_dim)
        self.relu2 = nn.ReLU()

        self.fc3 = nn.Linear(hidden_dim, hidden_dim)
        self.bn3 = nn.BatchNorm1d(hidden_dim)
        self.relu3 = nn.ReLU()

        self.fc4 = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        out = self.fc1(x)
        out = self.bn1(out)
        out = self.relu1(out)

        out = self.fc2(out)
        out = self.bn2(out)
        out = self.relu2(out)
        out = self.fc3(out)
        out = self.bn3(out)
        out = self.relu3(out)
        out = self.fc4(out)
        return out
```

مدل شامل چهار لایه کاملاً متصل (Fully Connected) است که به ترتیب با لایه‌های نرمال‌سازی و فعال‌سازی ترکیب شده‌اند. ساختار به صورت زیر است:

لایه اول (fc1): ورودی با ابعاد input_dim به خروجی با ابعاد hidden_dim نگاشت می‌شود.
Batch Normalization (bn1): خروجی لایه اول نرمال‌سازی می‌شود تا نوسانات میانگرادیانی کاهش یابد و روند آموزش پایدارتر شود. تابع فعال‌سازی ReLU (relu1): غیرخطی بودن به مدل اضافه می‌شود.
لایه دوم (fc2): نگاشتی از hidden_dim به hidden_dim دارد.
Batch Normalization (bn2): دوباره نرمال‌سازی.
ReLU (relu2): فعال‌سازی.
لایه سوم (fc3):
مشابه لایه دوم با نگاشت از hidden_dim به hidden_dim.
Batch Normalization (bn3) و ReLU (relu3): برای جلوگیری از overfitting و بهبود همگرایی.
لایه چهارم (fc4): نگاشتی از hidden_dim به output_dim دارد. این لایه بدون تابع فعال‌سازی است چون خروجی برای تابع خطای CrossEntropyLoss استفاده می‌شود که خودش عملیات Softmax را انجام می‌دهد.

```
input_dim = train_x.shape[1] # ابعاد ورودی برابر با تعداد ویژگی‌های داده
hidden_dim = 64
output_dim = len(torch.unique(torch.tensor(train_y))) # ابعاد خروجی برابر با تعداد کلاس‌ها

model = FeedforwardNeuralNetModel(input_dim, hidden_dim, output_dim)
criterion = nn.CrossEntropyLoss() # تابع خطا
learning_rate = 0.1 # بهینه‌ساز
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
```

آموزش مدل:

```
iter = 0
accuracy = 0

for epoch in range(num_epochs):
    for i, (inputs, labels) in enumerate(train_loader):
        inputs = inputs.requires_grad_()
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        iter += 1
```

داده‌های آموزشی در epochهای مختلف پردازش می‌شوند و در هر مرحله، گرادیان‌ها صفر شده، پیش‌بینی انجام می‌شود، خطا محاسبه و گرادیان‌ها به‌روزرسانی می‌شوند.

```
if iter % 500 == 0:
    correct = 0
    total = 0
    for inputs, labels in test_loader:
        inputs = inputs.requires_grad_()
        outputs = model(inputs)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    accuracy = 100 * correct / total

    print(f"Iteration: {iter}. Loss: {loss.item():.4f}. Accuracy: {accuracy:.2f}%")
```

هر ۵۰۰ تکرار یک بار دقت مدل بر روی داده‌های آزمایشی محاسبه و چاپ می‌شود.

```
if accuracy >= 80:
    print("accuracy >= 80")
else:
    print("accuracy < 80")
```

آیا دقت مدل به حداقل ۸۰٪ رسیده است یا نه

۱-ب:

```
torch.save(model.state_dict(), "model_checkpoint.pth")
print("model saved as a checkpoint.")

end_time = time.time()

print(f"Total Time: {end_time - start_time:.2f} seconds")
```

چکپوینت مدل تنها به وسیله ی یک پراسس ذخیره می شود و در نهایت زمان کل اجرا محاسبه می شود.

۱-ج:

با استفاده از دستور زیر فایل ها را پردازش می کنیم:

```
sbatch Q1_SMSC.sh && sbatch Q1_SMMC.sh && sbatch Q1_MMSC.sh && sbatch Q1_MMMC.sh
```

خروجی ها در فایل های لاگ ذخیره شده و ارور ها نیز به صورت جداگانه برای هر بخش فایل لاگ دارند:

/home/dmls/aqdam/tmrin4/1/				
Name	Size	Changed	Rights	Owner
		17/01/2025 08:30:59 ...	rw-r--r--	aqdam
train_data		17/01/2025 07:51:27 ...	rw-r--r--	aqdam
test_data		17/01/2025 07:51:21 ...	rw-r--r--	aqdam
Q1_SMSC.sh	1 KB	17/01/2025 08:01:58 ...	rw-r--r--	aqdam
Q1_SMSC.py	4 KB	17/01/2025 07:58:33 ...	rw-r--r--	aqdam
Q1_SMSC.log	1 KB	17/01/2025 08:26:32 ...	rw-r--r--	aqdam
Q1_SMMC.sh	1 KB	17/01/2025 08:01:51 ...	rw-r--r--	aqdam
Q1_SMMC.py	4 KB	17/01/2025 07:58:30 ...	rw-r--r--	aqdam
Q1_SMMC.log	1 KB	17/01/2025 08:26:03 ...	rw-r--r--	aqdam
Q1_MMSC.sh	1 KB	17/01/2025 08:06:02 ...	rw-r--r--	aqdam
Q1_MMSC.py	4 KB	17/01/2025 07:58:29 ...	rw-r--r--	aqdam
Q1_MMSC.log	1 KB	17/01/2025 08:28:00 ...	rw-r--r--	aqdam
Q1_MMMC.sh	1 KB	17/01/2025 08:05:50 ...	rw-r--r--	aqdam
Q1_MMMC.py	4 KB	17/01/2025 07:58:24 ...	rw-r--r--	aqdam
Q1_MMMC.log	1 KB	17/01/2025 08:28:03 ...	rw-r--r--	aqdam
model_checkpoint....	177 KB	17/01/2025 08:28:03 ...	rw-r--r--	aqdam
error_Q1_SMSC.log	0 KB	17/01/2025 08:06:03 ...	rw-r--r--	aqdam
error_Q1_SMMC.log	0 KB	17/01/2025 08:31:29 ...	rw-r--r--	aqdam
error_Q1_MMSC.log	0 KB	17/01/2025 08:26:48 ...	rw-r--r--	aqdam
error_Q1_MMMC.log	0 KB	17/01/2025 08:31:21 ...	rw-r--r--	aqdam

برای باز کردن هر فایل از دستور cat استفاده می کنیم. برای مثال:

```
cat Q1_SMSC.log
```

```
cat error_Q1_SMSC.log
```

یک ماشین و یک هسته:

```
#!/bin/bash
#SBATCH --job-name=Q1_SMSC #نام را تنظیم می کند
#SBATCH --output=Q1_SMSC.log #فایل خروجی را ذخیره می کند
#SBATCH --error=error_Q1_SMSC.log #فایل خطاها
```

```

#SBATCH --ntasks=1 #تعداد تسک ها
#SBATCH --cpus-per-task=1 #هر تسک یک cpu
#SBATCH --nodes=1 #تعداد نو دها
#SBATCH --partition=partition
#SBATCH --mem-per-cpu=1000
source /home/shared_files/pytorch_venv/bin/activate

echo '-----'
echo 'Nodelist: ' $SLURM_JOB_NODELIST #لیست نودهایی که کار روی آن ها اجرا می شود
echo "Number of nodes: " $SLURM_JOB_NUM_NODES #تعداد نودهایی که اختصاص داده شده اند
echo "Ntasks per node: " $SLURM_NTASKS #تعداد تسک های اختصاص داده شده به هر نود
echo '-----'

#این متغیرها برای راه اندازی و هماهنگی اجرای توزیع شده پایتورچ تنظیم می شوند
export MASTER_PORT=2048
export RENDEZVOUS_ID=$RANDOM
export WORLD_SIZE=2

#این بخش آدرس آبی نود اصلی را برای هماهنگی با سایر نودها تنظیم می کند
export MASTER_ADDR=$(hostname -I | awk '{print $1}')
echo "MASTER_ADDR:MASTER_PORT=$MASTER_ADDR:$MASTER_PORT"
echo '-----'

export OMP_NUM_THREADS=1

#اجرای اسکریپت پایتون را به صورت توزیع شده مدیریت می کند:
srun torchrun --nnodes=1 --nproc-per-node=1 --rdzv-id=$RENDEZVOUS_ID --
rdzv_backend=c10d --rdzv_endpoint=$MASTER_ADDR:$MASTER_PORT Q1_SMSC.py

```

```
(pytorch_venv) aqdam@raspberrypi-dml2:~/tmrin4/1 $ cat Q1_SMSC.log
-----
Nodelist: raspberrypi-dml1
Number of nodes: 1
Ntasks per node: 1
-----
MASTER_ADDR:MASTER_PORT=172.18.32.201:2048
-----
Iteration: 500. Loss: 0.8401. Accuracy: 81.43%
Iteration: 1000. Loss: 0.3236. Accuracy: 82.92%
Iteration: 1500. Loss: 0.2718. Accuracy: 83.27%
Iteration: 2000. Loss: 0.4794. Accuracy: 82.94%
Iteration: 2500. Loss: 0.3959. Accuracy: 83.34%
Iteration: 3000. Loss: 0.4385. Accuracy: 83.25%
accuracy >= 80
model saved as a checkpoint.
Total Time: 84.48 seconds
(pytorch_venv) aqdam@raspberrypi-dml2:~/tmrin4/1 $
```

یک ماشین و دو هسته:

```
#!/bin/bash
#SBATCH --job-name=Q1_SMMC
#SBATCH --output=Q1_SMMC.log
#SBATCH --error=error_Q1_SMMC.log
#SBATCH --ntasks=2
#SBATCH --cpus-per-task=1
#SBATCH --nodes=1
#SBATCH --partition=partition
#SBATCH --mem-per-cpu=1000

#فعال سازی محیط مجازی تضمین می کند که پکیج ها و وابستگی های مورد نیاز در دسترس باشند :فعال سازی محیط مجازی
source /home/shared_files/pytorch_venv/bin/activate
echo '-----'
echo 'Nodelist: ' $SLURM_JOB_NODELIST
echo "Number of nodes: " $SLURM_JOB_NUM_NODES
echo "Ntasks per node: " $SLURM_NTASKS
echo '-----'

export MASTER_PORT=2948
export RENDEZVOUS_ID=$RANDOM
```

```
export WORLD_SIZE=2

export MASTER_ADDR=$(hostname -I | awk '{print $1}')
echo "MASTER_ADDR:MASTER_PORT=$MASTER_ADDR:$MASTER_PORT"
echo '-----'

#Threads nums:
export OMP_NUM_THREADS=1

srun torchrun --nnodes=1 --nproc-per-node=2 --rdzv-id=$RENDEZVOUS_ID --
rdzv_backend=c10d --rdzv_endpoint=$MASTER_ADDR:$MASTER_PORT Q1_SMMC.py
```

--nnodes=1:

تعداد کل گره‌ها (Nodes) را مشخص می‌کند.

--nproc-per-node=2:

این پارامتر تعداد **Processe** های اجرا شده در هر گره را مشخص می‌کند. در این جا، مقدار آن برابر با ۲ است، بنابراین در هر گره دو فرآیند همزمان اجرا می‌شوند.

--rdzv-id=\$RENDEZVOUS_ID:

این پارامتر یک شناسه یکتا برای هماهنگی فرآیندها تعیین می‌کند. مقدار آن که به صورت تصادفی تولید شده است. این شناسه یکتا تضمین می‌کند که فرآیندهای مختلف در طول اجرای توزیع‌شده دچار تداخل نشوند و بتوانند به درستی هماهنگ شوند.

--rdzv_backend=c10d:

این پارامتر مشخص می‌کند که از چه بک‌اندی برای هماهنگی فرآیندها استفاده شود. مقدار **c10d** که بک‌اند پیش‌فرض **PyTorch** برای اجرای توزیع‌شده است، در اینجا استفاده شده است. این بک‌اند برای مدیریت ارتباطات بین فرآیندها در سیستم توزیع‌شده بهینه‌سازی شده و عملکرد بالایی دارد.

--rdzv_endpoint=\$MASTER_ADDR:\$MASTER_PORT:

این پارامتر آدرس و پورت گره اصلی را برای هماهنگی فرآیندها مشخص می‌کند. مقدار آدرس و پورت از متغیرهای **MASTER_ADDR** و **MASTER_PORT** گرفته می‌شود.

```
(pytorch_venv) aqdam@raspberrypi-dml2:~/tmrin4/1 $ cat Q1_SMMC.log
-----
Nodelist: raspberrypi-dml3
Number of nodes: 1
Ntasks per node: 2
-----
MASTER_ADDR:MASTER_PORT=172.18.32.203:2948
-----
Iteration: 500. Loss: 0.5929. Accuracy: 82.84%
Iteration: 500. Loss: 0.5787. Accuracy: 82.30%
Iteration: 1000. Loss: 0.3191. Accuracy: 83.16%
Iteration: 1000. Loss: 0.5222. Accuracy: 82.66%
Iteration: 1500. Loss: 0.3488. Accuracy: 83.44%Iteration: 1500. Loss: 0.3812. Accuracy: 82.72%

accuracy >= 80accuracy >= 80

model saved as a checkpoint.
Total Time: 52.56 seconds
model saved as a checkpoint.
Total Time: 52.55 seconds
(pytorch_venv) aqdam@raspberrypi-dml2:~/tmrin4/1 $ |
```

دو ماشین و یک هسته:

```
#!/bin/bash
#SBATCH --job-name=Q1_MMSC
#SBATCH --output=Q1_MMSC.log
#SBATCH --error=error_Q1_MMSC.log
#SBATCH --ntasks=2
#SBATCH --nodelist=raspberrypi-dml[0,2]
#SBATCH --cpus-per-task=1
#SBATCH --nodes=2
#SBATCH --partition=partition
#SBATCH --mem-per-cpu=1000
source /home/shared_files/pytorch_venv/bin/activate

echo '-----'
echo 'Nodelist: ' $SLURM_JOB_NODELIST
echo "Number of nodes: " $SLURM_JOB_NUM_NODES
echo "Ntasks per node: " $SLURM_NTASKS
echo '-----'

export MASTER_PORT=4820
export RENDEZVOUS_ID=$RANDOM
export WORLD_SIZE=4

export MASTER_ADDR=$(hostname -I | awk '{print $1}')
echo "MASTER_ADDR:MASTER_PORT=$MASTER_ADDR:$MASTER_PORT"
echo '-----'

export OMP_NUM_THREADS=1

srun torchrun --nnodes=2 --nproc-per-node=1 --rdzv-id=$RENDEZVOUS_ID --
rdzv_backend=c10d --rdzv_endpoint=$MASTER_ADDR:$MASTER_PORT Q1_MMSC.py
```

```
(pytorch_venv) aqdam@raspberrypi-dml2:~/tmrin4/1 $ cat Q1_MMSC.log
-----
Nodelist: raspberrypi-dml[0,2]
Number of nodes: 2
Ntasks per node: 2
-----
MASTER_ADDR:MASTER_PORT=172.18.32.200:4820
-----
Iteration: 500. Loss: 0.6475. Accuracy: 82.88%
Iteration: 500. Loss: 0.5774. Accuracy: 82.72%
Iteration: 1000. Loss: 0.4875. Accuracy: 82.92%
Iteration: 1000. Loss: 0.4018. Accuracy: 83.98%
Iteration: 1500. Loss: 0.4593. Accuracy: 82.96%
accuracy >= 80
Iteration: 1500. Loss: 0.4317. Accuracy: 83.70%
accuracy >= 80
model saved as a checkpoint.
Total Time: 63.00 seconds
model saved as a checkpoint.
Total Time: 63.41 seconds
(pytorch_venv) aqdam@raspberrypi-dml2:~/tmrin4/1 $ |
```

دو ماشین و دو هسته:

```
#!/bin/bash
#SBATCH --job-name=Q1_MMMC
#SBATCH --output=Q1_MMMC.log
#SBATCH --error=error_Q1_MMMC.log
#SBATCH --ntasks=4
#SBATCH --cpus-per-task=1
#SBATCH --nodelist=raspberrypi-dml[0,2]
#SBATCH --nodes=2
#SBATCH --partition=partition
#SBATCH --mem-per-cpu=1000
source /home/shared_files/pytorch_venv/bin/activate

echo '-----'
echo 'Nodelist: ' $SLURM_JOB_NODELIST
echo "Number of nodes: " $SLURM_JOB_NUM_NODES
echo "Ntasks per node: " $SLURM_NTASKS
echo '-----'

export MASTER_PORT=2940
export RENDEZVOUS_ID=$RANDOM
```

```
export WORLD_SIZE=4

export MASTER_ADDR=$(hostname -I | awk '{print $1}')
echo "MASTER_ADDR:MASTER_PORT=$MASTER_ADDR:$MASTER_PORT"
echo '-----'

export OMP_NUM_THREADS=1

srun torchrun --nnodes=2 --nproc-per-node=2 --rdzv-id=$RENDEZVOUS_ID --
rdzv_backend=c10d --rdzv_endpoint=$MASTER_ADDR:$MASTER_PORT Q1_MMMC.py
```

```
total time: 66.12 seconds
(pytorch_venv) aqdam@raspberrypi-dml2:~/tmrin4/1 $ cat Q1_MMMC.log
-----
Nodelist: raspberrypi-dml[0,2]
Number of nodes: 2
Ntasks per node: 4
-----
MASTER_ADDR:MASTER_PORT=172.18.32.200:2940
-----
Iteration: 500. Loss: 0.2638. Accuracy: 83.48%
Iteration: 500. Loss: 0.3354. Accuracy: 83.64%
Iteration: 500. Loss: 0.4629. Accuracy: 82.40%
Iteration: 500. Loss: 0.3998. Accuracy: 84.00%
accuracy >= 80accuracy >= 80

accuracy >= 80
accuracy >= 80
model saved as a checkpoint.
Total Time: 66.80 seconds
model saved as a checkpoint.
Total Time: 66.82 seconds
model saved as a checkpoint.
Total Time: 66.89 seconds
model saved as a checkpoint.
Total Time: 66.89 seconds
(pytorch_venv) aqdam@raspberrypi-dml2:~/tmrin4/1 $ |
```

```
(pytorch_venv) aqdam@raspberrypi-dml2:~/tmrin4/1 $ cat error_Q1_MMMC.log && cat error_Q1_MMSC.log && cat error_Q1_SMMC.l
og && cat error_Q1_SMSC.log
(pytorch_venv) aqdam@raspberrypi-dml2:~/tmrin4/1 $ |
```

۱-۵:

حالت اول: یک نود و یک هسته

زمان آموزش: ۸۴.۴۸ ثانیه (بیشترین زمان)

بیشترین دقت: ۸۳.۳۴٪

از آنجا که فقط یک هسته درگیر است، تمام عملیات‌ها به‌صورت متوالی انجام می‌شوند.

این حالت به دلیل کمترین منابع محاسباتی، بیشترین زمان آموزش را دارد.

دقت قابل قبولی دارد اما از سایر حالت‌ها پایین‌تر است.

حالت دوم: یک نود و دو هسته

زمان آموزش: ۵۲.۵۶ ثانیه (کمترین زمان)

بیشترین دقت: ۸۳.۴۴٪

افزایش تعداد هسته‌ها به ۲ هسته باعث توزیع بهتر بار پردازشی و کاهش قابل توجه زمان آموزش شده است. کاهش زمان نسبت به

حالت اول بسیار محسوس است ($\sim 38\%$ سریع‌تر). دقت مدل کمی بهبود یافته اما تفاوت چندانی ندارد.

حالت سوم: دو نود و دو هسته

زمان آموزش: ۶۳.۴۱ ثانیه

بیشترین دقت: ۸۳.۹۸٪

با اضافه شدن یک نود دیگر، منابع پردازشی افزایش یافته است.

با این حال، به دلیل هزینه‌های همگام‌سازی (Synchronization Overhead) بین نودها، کاهش زمان آموزش نسبت به حالت

دوم چندان محسوس نیست.

دقت مدل بهبود یافته و به نزدیک ۸۴٪ رسیده است.

حالت چهارم: دو نود و چهار هسته

زمان آموزش: ۶۶.۸۹ ثانیه

بیشترین دقت: ۸۴.۰۰٪

با افزایش تعداد هسته‌ها به چهار هسته در دو نود، انتظار می‌رفت زمان آموزش کاهش بیشتری یابد. اما به دلیل افزایش هزینه‌های

همگام‌سازی بین نودها و هسته‌ها، زمان آموزش کمی افزایش یافته است. دقت مدل به بالاترین مقدار یعنی ۸۴.۰۰٪ رسیده است.

مقایسه زمان آموزش:

بیشترین کاهش زمان بین حالت اول و دوم مشاهده شد. افزایش نودها از ۱ به ۲ باعث کاهش زمان نشد و حتی در حالت دو نود و

چهار هسته، زمان افزایش یافت.

دلیل این افزایش، همگام‌سازی پردازش‌ها بین نودها است که زمان اضافی مصرف می‌کند.

مقایسه دقت مدل:

دقت مدل در همه حالت‌ها از ۸۳٪ فراتر رفت. بیشترین دقت مربوط به دو نود و چهار هسته بود که به ۸۴.۰۰٪ رسید.

بهبود دقت با افزایش منابع محاسباتی قابل مشاهده است اما این بهبود جزئی است.

بنابراین:

تأثیر افزایش تعداد هسته‌ها بر زمان آموزش

افزایش تعداد هسته‌ها در یک نود تأثیر قابل توجهی در کاهش زمان آموزش داشت. به‌طور خاص، با افزایش تعداد هسته‌ها از ۱ به ۲ در یک نود، زمان آموزش از ۸۴.۴۸ ثانیه به ۵۲.۵۶ ثانیه کاهش یافت. این کاهش زمان نشان می‌دهد که توزیع بار پردازشی بین هسته‌ها به بهینه‌تر شدن فرآیند آموزش کمک می‌کند. با این حال، افزایش بیشتر تعداد هسته‌ها و نودها به دلیل هزینه‌های همگام‌سازی (Synchronization Overhead) نتوانست به همان اندازه مؤثر باشد.

تأثیر افزایش نودها بر زمان و دقت مدل

افزایش تعداد نودها از ۱ به ۲ در کنار افزایش تعداد هسته‌ها، اگرچه باعث بهبود دقت مدل شد، اما در کاهش زمان آموزش چندان مؤثر نبود. در حالت دو نود و دو هسته، زمان آموزش ۶۳.۴۱ ثانیه بود که نسبت به حالت یک نود و دو هسته افزایش یافت. این افزایش زمان به دلیل نیاز به همگام‌سازی پردازش‌ها بین نودها است که باعث افزایش تأخیر در پردازش می‌شود. با این حال، دقت مدل در این حالت به ۸۳.۹۸٪ افزایش یافت.

بالاترین دقت با افزایش منابع

بیشترین دقت مدل در حالت دو نود و چهار هسته با ۸۴.۰۰٪ به دست آمد. این افزایش دقت نشان‌دهنده تأثیر مثبت استفاده از منابع بیشتر بر عملکرد مدل است. با این حال، زمان آموزش در این حالت به ۶۶.۸۹ ثانیه رسید که نسبت به حالت یک نود و دو هسته افزایش داشت. این نتیجه نشان می‌دهد که افزایش منابع محاسباتی همواره باعث بهبود عملکرد زمانی نمی‌شود و ممکن است به دلیل افزایش زمان همگام‌سازی کارایی کاهش یابد.

سوال ۲:

تفاوت‌ها با کد قبلی در طول کد گفته شده اند:

```
#1- الف
import time
start_time = time.time()
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset
from accelerate import Accelerator
```

این خط یک شیء از کلاس Accelerator ایجاد می‌کند.

پارامتر mixed_precision مشخص می‌کند که آیا از دقت مخلوط (Mixed Precision) برای محاسبات استفاده شود یا نه.

در اینجا مقدار پیشفرض "no" تنظیم شده است، یعنی محاسبات با دقت کامل (FP32) انجام می‌شوند.

این نمونه مدیریت تمام عملیات مرتبط با توزیع داده‌ها، مدل، و بهینه‌ساز بین دستگاه‌های شتاب‌دهنده را بر عهده می‌گیرد.

```
accelerator = Accelerator()
test_x = np.load('./test_data/test_x.npy').astype(np.float32)
test_y = np.load('./test_data/test_y.npy').astype(np.float32)
test_dataset = TensorDataset(torch.tensor(test_x).float(),
torch.tensor(test_y).long())
```

```

train_x = np.load("./train_data/train_x.npy").astype(np.float32)
train_y = np.load("./train_data/train_y.npy").astype(np.float32)
train_dataset = TensorDataset(torch.tensor(train_x).float(),
                                torch.tensor(train_y).long())

batch_size = 100
n_iters = 3000
num_epochs = n_iters / (len(train_dataset) / batch_size)
num_epochs = int(num_epochs)

train_loader = torch.utils.data.DataLoader(dataset=train_dataset,
                                             batch_size=batch_size,
                                             shuffle=True)

test_loader = torch.utils.data.DataLoader(dataset=test_dataset,
                                           batch_size=batch_size,
                                           shuffle=False)

class FeedforwardNeuralNetModel(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super(FeedforwardNeuralNetModel, self).__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.bn1 = nn.BatchNorm1d(hidden_dim)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.bn2 = nn.BatchNorm1d(hidden_dim)
        self.relu2 = nn.ReLU()

        self.fc3 = nn.Linear(hidden_dim, hidden_dim)
        self.bn3 = nn.BatchNorm1d(hidden_dim)
        self.relu3 = nn.ReLU()

        self.fc4 = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        out = self.fc1(x)
        out = self.bn1(out)
        out = self.relu1(out)

        out = self.fc2(out)
        out = self.bn2(out)
        out = self.relu2(out)

        out = self.fc3(out)
        out = self.bn3(out)

```

```

        out = self.relu3(out)
        out = self.fc4(out)
        return out

input_dim = train_x.shape[1]
hidden_dim = 64
output_dim = len(torch.unique(torch.tensor(train_y)))

```

متد prepare از شیء Accelerator اجزای مختلف (مانند مدل، بهینه‌ساز و DataLoader ها) را برای اجرا با دستگاه‌های شتاب‌دهنده آماده می‌کند.

```

model = FeedforwardNeuralNetModel(input_dim, hidden_dim, output_dim)
criterion = nn.CrossEntropyLoss()
learning_rate = 0.1
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
model, optimizer, train_loader, test_loader = accelerator.prepare(model,
optimizer, train_loader, test_loader)

iter = 0
accuracy = 0

for epoch in range(num_epochs):
    for i, (inputs, labels) in enumerate(train_loader):
        inputs = inputs.requires_grad_()
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)

```

به جای استفاده مستقیم از `loss.backward()` از `accelerator.backward(loss)` استفاده شده است. این متد سازگار با محاسبات توزیع‌شده و Mixed Precision است و از بروز خطاهای ناشی از تفاوت دقت در پردازش جلوگیری می‌کند.

```

accelerator.backward(loss)
optimizer.step()

iter += 1

if iter % 500 == 0:
    correct = 0
    total = 0

```

```

        for inputs, labels in test_loader:
            inputs = inputs.requires_grad_()
            outputs = model(inputs)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    accuracy = 100 * correct / total

    print(f"Iteration: {iter}. Loss: {loss.item():.4f}. Accuracy:
{accuracy:.2f}%")

if accuracy >= 80:
    print("accuracy >= 80")
else:
    print("accuracy < 80")
#1-ب
torch.save(model.state_dict(), "model_checkpoint.pth")
print("model saved as a checkpoint.")

end_time = time.time()

print(f"Total Time: {end_time - start_time:.2f} seconds")

```

یک ماشین و یک هسته:

```

#!/bin/bash
#SBATCH --job-name=Q2_SMSC
#SBATCH --output=Q2_SMSC.log
#SBATCH --error=error_Q2_SMSC.log
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=1
#SBATCH --nodes=1
#SBATCH --partition=partition
#SBATCH --mem-per-cpu=1000

echo '-----'
echo 'Nodelist: ' $SLURM_JOB_NODELIST
echo "Number of nodes: " $SLURM_JOB_NUM_NODES
echo "Ntasks per node:" $SLURM_NTASKS

```

```
echo '-----'
echo "SLURM_NTASKS:" $SLURM_NTASKS
echo "SLURM_NTASKS_PER_NODE:" $SLURM_NTASKS_PER_NODE
echo "SLURM_JOB_NUM_NODES:" $SLURM_JOB_NUM_NODES

export MASTER_PORT=5048
export WORLD_SIZE=1
export MASTER_ADDR=$(hostname)

export OMP_NUM_THREADS=1

source /home/shared_files/pytorch_venv/bin/activate

srun accelerate launch \
  --mixed_precision=no \
  --dynamo_backend=no \
  --num_processes=1 \
  --num_machines=1 \
  --machine_rank=0 \
  --main_process_ip=$MASTER_ADDR \
  --main_process_port=$MASTER_PORT \
  Q2_SMSC.py
```

به جای torchrun از accelerate استفاده شده است. مقدار دیفالت mixed_precision روی fp32 می باشد.

```

aqdam@raspberrypi-dml2:~/tmrin4/2 $ cat Q2_SMSC.log
-----
Nodelist: raspberrypi-dml0
Number of nodes: 1
Ntasks per node: 1
-----
SLURM_NTASKS: 1
SLURM_NTASKS_PER_NODE: 1
SLURM_JOB_NUM_NODES: 1
Iteration: 500. Loss: 0.6882. Accuracy: 81.13%
Iteration: 1000. Loss: 0.4442. Accuracy: 82.82%
Iteration: 1500. Loss: 0.4851. Accuracy: 83.23%
Iteration: 2000. Loss: 0.3519. Accuracy: 83.38%
Iteration: 2500. Loss: 0.3419. Accuracy: 83.15%
Iteration: 3000. Loss: 0.3917. Accuracy: 82.89%
accuracy >= 80
Total Time: 62.80 seconds
aqdam@raspberrypi-dml2:~/tmrin4/2 $ cat error_Q2_SMSC.log
aqdam@raspberrypi-dml2:~/tmrin4/2 $ |

```

بدون ارور

این تنظیم ساده‌ترین حالت بوده و به دلیل استفاده محدود از منابع پردازشی، زمان اجرا نسبتاً طولانی است. دقت مدل به تدریج افزایش یافته و در پایان آموزش به مقدار قابل قبولی رسیده است، اما زمان اجرا نشان‌دهنده محدودیت قابل توجه در بهره‌وری منابع است.

یک ماشین و دو هسته:

```
-----  
MASTER_ADDR:MASTER_PORT=raspberrypi-dml0:2958  
-----  
Iteration: 500. Loss: 0.5931. Accuracy: 81.16%  
Iteration: 1000. Loss: 0.4330. Accuracy: 82.59%  
Iteration: 1500. Loss: 0.3426. Accuracy: 82.77%  
Iteration: 2000. Loss: 0.4318. Accuracy: 82.88%  
Iteration: 2500. Loss: 0.3302. Accuracy: 83.39%  
Iteration: 3000. Loss: 0.4031. Accuracy: 82.95%  
accuracy >= 80  
model saved as a checkpoint.  
Total Time: 62.05 seconds
```

مان اجرای مدل به ۶۲.۰۵ ثانیه کاهش یافت که نسبت به حالت اول ۰.۷۵ ثانیه سریع تر است. حداکثر دقت مدل نیز ۸۳.۳۹٪ بود که تفاوت ناچیزی با حالت قبلی دارد (افزایش ۰.۰۱٪). این نتایج نشان می‌دهند که افزایش تعداد هسته‌ها، اگرچه زمان اجرا را کاهش می‌دهد، تأثیر چندانی بر دقت مدل ندارد.

دو ماشین و یک هسته:

```
7/home/umis/aquari/thin4/17QT_MVSC.log - aquari@172.18
-----
Nodelist: raspberrypi-dml[0,2]
Number of nodes: 2
Ntasks per node: 2
-----
MASTER_ADDR:MASTER_PORT=172.18.32.200:4820
-----
Iteration: 500. Loss: 0.6475. Accuracy: 82.88%
Iteration: 500. Loss: 0.5774. Accuracy: 82.72%
Iteration: 1000. Loss: 0.4875. Accuracy: 82.92%
Iteration: 1000. Loss: 0.4018. Accuracy: 83.98%
Iteration: 1500. Loss: 0.4593. Accuracy: 82.96%
accuracy >= 80
Iteration: 1500. Loss: 0.4317. Accuracy: 83.70%
accuracy >= 80
model saved as a checkpoint.
Total Time: 63.00 seconds
model saved as a checkpoint.
Total Time: 63.41 seconds
```

زمان اجرای مدل به ۶۳.۴۱ ثانیه افزایش یافت که نسبت به حالت دوم کمی بیشتر است. با این حال، دقت مدل به ۸۳.۹۸٪ رسید که بالاترین مقدار دقت در میان تمام تنظیمات است. این نتیجه نشان می‌دهد که استفاده از چند ماشین می‌تواند دقت مدل را بهبود بخشد، حتی اگر منجر به افزایش زمان اجرا شود. این تأخیر احتمالاً به دلیل اضافی هماهنگی بین ماشین‌ها بوده است.

دو ماشین و دو هسته:

در این حالت، از دو ماشین و دو هسته استفاده شد که منجر به بهبود چشمگیر زمان اجرا شد. زمان اجرای مدل برابر با ۵۲.۰۶ ثانیه بود که سریع‌ترین زمان اجرا در میان تمام تنظیمات است. حداکثر دقت مدل نیز ۸۳.۹۰٪ بود که تفاوت ناچیزی با حالت سوم دارد (کاهش ۰.۰۸٪). این حالت بهترین تعادل بین دقت و زمان اجرا را ارائه داد، زیرا هم دقت بالا و هم زمان اجرای کوتاه را فراهم کرده است.

```
/home/dmls/aqdam/tmrin4/2/Q2_MMMC.log - aqdam@172.18.32.200 - Editor - WinSCP
Encoding Color
-----
Running on two machines with one core each
Nodelist: raspberrypi-dml[0-1]
Number of nodes: 2
Ntasks per node: 4
-----
MASTER_ADDR:MASTER_PORT=172.18.32.200:48557
-----
DistributedType.MULTI_CPUDistributedType.MULTI_CPU
DistributedType.MULTI_CPUDistributedType.MULTI_CPU
Iteration: 500. Loss: 0.5331. Accuracy: 82.14%Iteration: 500. Loss: 0.5953. Accuracy: 81.84%
Iteration: 1000. Loss: 0.4555. Accuracy: 82.78%Iteration: 1000. Loss: 0.3411. Accuracy: 83.26%
Iteration: 1500. Loss: 0.4076. Accuracy: 83.90%Iteration: 1500. Loss: 0.4771. Accuracy: 83.08%
accuracy >= 80
accuracy >= 80
model saved as a checkpoint.
model saved as a checkpoint.
Total Time: 52.00 seconds
Total Time: 52.06 seconds
```

مقایسه این چهار حالت نشان می‌دهد که:

- افزایش تعداد هسته‌ها زمان اجرا را کاهش می‌دهد اما تأثیر چندانی بر دقت مدل ندارد. این موضوع در حالت‌های اول و دوم و همچنین حالت‌های سوم و چهارم مشهود است.
- افزایش تعداد ماشین‌ها دقت مدل را بهبود می‌بخشد، اما ممکن است زمان اجرا را نیز افزایش دهد. این موضوع در مقایسه حالت‌های ۱ و ۳ قابل مشاهده است.
- ترکیب استفاده از دو ماشین و دو هسته (حالت ۴) بهترین تعادل را بین کاهش زمان اجرا و حفظ دقت مدل ایجاد می‌کند.

۳:

برای `mixed_precision="no"` باید کد زیر اضافه شود:

```
accelerator = Accelerator(mixed_precision="no")
```

ت۴: حالت یک ماشین و یک هس

```
#!/bin/bash
#SBATCH --job-name=Q3no
```

```
#SBATCH --output=Q3no.log
#SBATCH --error=error_Q3no.log
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --nodes=1
#SBATCH --partition=partition
#SBATCH --mem-per-cpu=1000

echo '-----'
echo 'Nodelist: ' $SLURM_JOB_NODELIST
echo "Number of nodes: " $SLURM_JOB_NUM_NODES
echo "Ntasks per node: " $SLURM_NTASKS
echo '-----'

export MASTER_PORT=2048
export RENDEZVOUS_ID=$RANDOM
export WORLD_SIZE=1

export MASTER_ADDR=$(scontrol show hostnames "$SLURM_JOB_NODELIST" | head -n 1)
echo "MASTER_ADDR:MASTER_PORT=$MASTER_ADDR:$MASTER_PORT"
echo '-----'

export OMP_NUM_THREADS=1
source /home/shared_files/pytorch_venv/bin/activate

accelerate launch \
  --num_processes=1 \
  --num_machines=1 \
  --machine_rank=0 \
  --main_process_ip=$MASTER_ADDR \
  --main_process_port=$MASTER_PORT \
  Q3no.py
```



```

echo '-----'

export OMP_NUM_THREADS=1
source /home/shared_files/pytorch_venv/bin/activate

accelerate launch \
  --num_processes=1 \
  --num_machines=1 \
  --machine_rank=0 \
  --main_process_ip=$MASTER_ADDR \
  --main_process_port=$MASTER_PORT \
  Q3no.py

```

```

-----
Nodelist:  raspberrypi-dml0
Number of nodes:  1
Ntasks per node:  1
-----
MASTER_ADDR:MASTER_PORT=raspberrypi-dml0:2948
-----
Iteration: 500. Loss: 0.5091. Accuracy: 81.74%
Iteration: 1000. Loss: 0.5156. Accuracy: 82.77%
Iteration: 1500. Loss: 0.6748. Accuracy: 83.37%
Iteration: 2000. Loss: 0.5192. Accuracy: 82.86%
Iteration: 2500. Loss: 0.4110. Accuracy: 82.87%
Iteration: 3000. Loss: 0.3266. Accuracy: 82.90%
Total Time: 80.54 seconds

```

برای `mixed_precision="fp16"` باید کد زیر اضافه شود:

```

accelerator = Accelerator(mixed_precision="fp16")

```

یک ماشین و یک هسته:

```

#!/bin/bash
#SBATCH --job-name=Q3fp16
#SBATCH --output=Q3fp16.log
#SBATCH --error=error_Q3fp16.log
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=2

```

```
#SBATCH --nodes=1
#SBATCH --partition=partition
#SBATCH --mem-per-cpu=1000

echo '-----'
echo 'Nodelist: ' $SLURM_JOB_NODELIST
echo "Number of nodes: " $SLURM_JOB_NUM_NODES
echo "Ntasks per node: " $SLURM_NTASKS
echo '-----'

export MASTER_PORT=2099
export RENDEZVOUS_ID=$RANDOM
export WORLD_SIZE=1

export MASTER_ADDR=$(scontrol show hostnames "$SLURM_JOB_NODELIST" | head -n 1)
echo "MASTER_ADDR:MASTER_PORT=$MASTER_ADDR:$MASTER_PORT"
echo '-----'

export OMP_NUM_THREADS=2
source /home/shared_files/pytorch_venv/bin/activate

accelerate launch \
  --num_processes=1 \
  --num_machines=1 \
  --machine_rank=0 \
  --main_process_ip=$MASTER_ADDR \
  --main_process_port=$MASTER_PORT \
  Q3fp16.py
```

```
/home/dmls/aqdam/tmrin4/3/Q3fp16.log – aqdam@172.18.32.200 – Editor – WinSCP

-----
Nodelist:  raspberrypi-dml0
Number of nodes:  1
Ntasks per node:  1
-----
MASTER_ADDR:MASTER_PORT=raspberrypi-dml0:2099
-----
Iteration: 500. Loss: 0.4606. Accuracy: 81.59%
Iteration: 1000. Loss: 0.5039. Accuracy: 82.76%
Iteration: 1500. Loss: 0.3294. Accuracy: 83.34%
Iteration: 2000. Loss: 0.1883. Accuracy: 83.26%
Iteration: 2500. Loss: 0.3746. Accuracy: 83.44%
Iteration: 3000. Loss: 0.2838. Accuracy: 83.48%
Total Time: 101.87 seconds
```

یک ماشین و دو هسته:

```
#!/bin/bash
#SBATCH --job-name=Q3fp1612
#SBATCH --output=Q3fp1612.log
#SBATCH --error=error_Q3fp1612.log
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=2
#SBATCH --nodes=1
#SBATCH --partition=partition
#SBATCH --mem-per-cpu=1000

echo '-----'
echo 'Nodelist: ' $SLURM_JOB_NODELIST
echo "Number of nodes: " $SLURM_JOB_NUM_NODES
echo "Ntasks per node: " $SLURM_NTASKS
echo '-----'

export MASTER_PORT=9948
export RENDEZVOUS_ID=$RANDOM
export WORLD_SIZE=1

export MASTER_ADDR=$(scontrol show hostnames "$SLURM_JOB_NODELIST" | head -n 1)
echo "MASTER_ADDR:MASTER_PORT=$MASTER_ADDR:$MASTER_PORT"
echo '-----'
```

```
export OMP_NUM_THREADS=2
source /home/shared_files/pytorch_venv/bin/activate

accelerate launch \
  --num_processes=1 \
  --num_machines=1 \
  --machine_rank=0 \
  --main_process_ip=$MASTER_ADDR \
  --main_process_port=$MASTER_PORT \
  Q3fp16.py
```

 /home/dmls/aqdam/tmrin4/3/Q3fp1612.log – aqdam@172.18.32.200 – Editor – WinSCP

 Encoding ▾ ☐ Color ▾  

```
-----
Modelist: raspberrypi-dml2
Number of nodes: 1
Ntasks per node: 1
-----
MASTER_ADDR:MASTER_PORT=raspberrypi-dml2:9948
-----
Iteration: 500. Loss: 0.5512. Accuracy: 81.55%
Iteration: 1000. Loss: 0.4872. Accuracy: 82.82%
Iteration: 1500. Loss: 0.3359. Accuracy: 83.39%
Iteration: 2000. Loss: 0.4799. Accuracy: 83.41%
Iteration: 2500. Loss: 0.4243. Accuracy: 83.18%
Iteration: 3000. Loss: 0.2848. Accuracy: 83.39%
Total Time: 59.41 seconds
```

برای `mixed_precision="bf16"` باید کد زیر اضافه شود:

```
accelerator = Accelerator(mixed_precision="bf16")
```

یک ماشین و یک هسته:

```
#!/bin/bash
#SBATCH --job-name=Q3bf16
#SBATCH --output=Q3bf16.log
#SBATCH --error=error_Q3bf16.log
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --nodes=1
#SBATCH --partition=partition
```

```
#SBATCH --mem-per-cpu=1000

echo '-----'
echo 'Nodelist: ' $SLURM_JOB_NODELIST
echo "Number of nodes: " $SLURM_JOB_NUM_NODES
echo "Ntasks per node: " $SLURM_NTASKS
echo '-----'

export MASTER_PORT=2592
export RENDEZVOUS_ID=$RANDOM
export WORLD_SIZE=1

export MASTER_ADDR=$(scontrol show hostnames "$SLURM_JOB_NODELIST" | head -n 1)
echo "MASTER_ADDR:MASTER_PORT=$MASTER_ADDR:$MASTER_PORT"
echo '-----'

export OMP_NUM_THREADS=1

source /home/shared_files/pytorch_venv/bin/activate

accelerate launch \
  --num_processes=1 \
  --num_machines=1 \
  --machine_rank=0 \
  --main_process_ip=$MASTER_ADDR \
  --main_process_port=$MASTER_PORT \
  Q3bf16.py
```

```
/home/dmls/aqdam/tmrin4/3/Q3bf16.log – aqdam@172.18.32.200 – Editor – WinSCP

-----
Nodelist:  raspberrypi-dml1
Number of nodes:  1
Ntasks per node:  1
-----
MASTER_ADDR:MASTER_PORT=raspberrypi-dml1:2592
-----
Iteration: 500. Loss: 0.5656. Accuracy: 81.53%
Iteration: 1000. Loss: 0.4484. Accuracy: 82.86%
Iteration: 1500. Loss: 0.5370. Accuracy: 83.15%
Iteration: 2000. Loss: 0.3438. Accuracy: 83.16%
Iteration: 2500. Loss: 0.2173. Accuracy: 83.66%
Iteration: 3000. Loss: 0.3371. Accuracy: 83.23%
Total Time: 180.66 seconds
```

یک ماشین و دو هسته:

```
#!/bin/bash
#SBATCH --job-name=Q3bf1612
#SBATCH --output=Q3bf1612.log
#SBATCH --error=error_Q3bf1612.log
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=2
#SBATCH --nodes=1
#SBATCH --partition=partition
#SBATCH --mem-per-cpu=1000

echo '-----'
echo 'Nodelist: ' $SLURM_JOB_NODELIST
echo "Number of nodes: " $SLURM_JOB_NUM_NODES
echo "Ntasks per node: " $SLURM_NTASKS
echo '-----'

export MASTER_PORT=2998
export RENDEZVOUS_ID=$RANDOM
export WORLD_SIZE=1

export MASTER_ADDR=$(scontrol show hostnames "$SLURM_JOB_NODELIST" | head -n 1)
echo "MASTER_ADDR:MASTER_PORT=$MASTER_ADDR:$MASTER_PORT"
echo '-----'
```

```
export OMP_NUM_THREADS=1

source /home/shared_files/pytorch_venv/bin/activate

accelerate launch \
  --num_processes=1 \
  --num_machines=1 \
  --machine_rank=0 \
  --main_process_ip=$MASTER_ADDR \
  --main_process_port=$MASTER_PORT \
  Q3bf16.py
```

```

/home/dmls/aqdam/tmrin4/3/Q3bf1612.log – aqdam@172.18.32.200 – Editor – WinSCP
-----
Modelist:  raspberrypi-dml3
Number of nodes:  1
Ntasks per node:  1
-----
MASTER_ADDR:MASTER_PORT=raspberrypi-dml3:2998
-----
Iteration: 500. Loss: 0.6402. Accuracy: 80.96%
Iteration: 1000. Loss: 0.7893. Accuracy: 82.52%
Iteration: 1500. Loss: 0.3772. Accuracy: 82.98%
Iteration: 2000. Loss: 0.3096. Accuracy: 83.11%
Iteration: 2500. Loss: 0.2980. Accuracy: 83.08%
Iteration: 3000. Loss: 0.2523. Accuracy: 83.13%
Total Time: 146.91 seconds

```

تحلیل و مقایسه عملکرد مدل در حالت‌های مختلف

۱. یک ماشین با یک هسته و حالت `mixed_precision=no` در این حالت، مدل با استفاده از یک ماشین و یک هسته و بدون استفاده از Mixed Precision (دقت ۳۲ بیتی) اجرا شده است. این حالت معمولاً دارای بالاترین دقت است زیرا محاسبات با دقت کامل انجام می‌شود. با این حال، به دلیل استفاده از تنها یک هسته و عدم بهینه‌سازی حافظه، زمان آموزش طولانی است و منابع سخت‌افزاری به‌صورت کامل مصرف می‌شود.

۲. یک ماشین با یک هسته و حالت `mixed_precision=fp16`

با استفاده از دقت ۱۶ بیتی (FP16)، حجم محاسبات کاهش یافته و مصرف حافظه کمتر شده است. در نتیجه، زمان آموزش کاهش یافته اما ممکن است دقت مدل کمی کاهش پیدا کند. این حالت برای GPUهای مدرن که از FP16 پشتیبانی می‌کنند، مناسب است. با این وجود، به دلیل استفاده از یک هسته، همچنان پردازش محدود است.

۳. یک ماشین با یک هسته و حالت `mixed_precision=bf16` در این حالت، از دقت (bf16) BFloat16 استفاده شده است. این نوع دقت با حفظ پایداری عددی بهتر از FP16، عملکردی مشابه دقت ۳۲ بیتی ارائه می‌دهد. در نتیجه، زمان آموزش نسبت به حالت پیش‌فرض (no) کاهش یافته و دقت مدل نیز تقریباً ثابت مانده است. این حالت برای پردازنده‌ها و GPUهایی که از BFloat16 پشتیبانی می‌کنند، مناسب است.

۴. یک ماشین با دو هسته و حالت `mixed_precision=no` افزایش تعداد هسته‌ها از یک به دو هسته باعث افزایش سرعت پردازش و کاهش زمان آموزش شده است. با این حال، چون Mixed Precision غیرفعال است، مصرف حافظه بالا باقی مانده است. در این حالت، دقت مدل بالا است و سرعت آموزش نسبت به حالت یک هسته بهبود یافته است.

۵. یک ماشین با دو هسته و حالت `mixed_precision=fp16` با ترکیب دو هسته و استفاده از FP16، بیشترین کاهش زمان آموزش مشاهده شده است. این ترکیب به دلیل توزیع بهتر بار محاسباتی و بهینه‌سازی حافظه، بهترین عملکرد زمانی را ارائه می‌دهد. اما ممکن است دقت مدل کمی کاهش پیدا کند. این حالت برای مواقعی که سرعت آموزش مهم‌تر از دقت نهایی است، پیشنهاد می‌شود.

۶. یک ماشین با دو هسته و حالت `mixed_precision=bf16` استفاده از BFloat16 با دو هسته باعث ایجاد تعادل بین سرعت آموزش و دقت مدل شده است. زمان آموزش در این حالت تقریباً به اندازه حالت fp16 کاهش یافته، اما دقت مدل پایدارتر باقی مانده است. این ترکیب برای شرایطی که نیاز به پایداری عددی و همچنین افزایش سرعت وجود دارد، گزینه مناسبی است.

بنابراین:

- حالت پیش‌فرض (no) بیشترین دقت را دارد اما زمان آموزش طولانی است.
- حالت fp16 سریع‌ترین آموزش را فراهم می‌کند اما ممکن است باعث کاهش جزئی دقت شود.
- حالت bf16 تعادل مناسبی بین سرعت آموزش و دقت مدل ایجاد می‌کند.
- افزایش تعداد هسته‌ها از یک به دو تأثیر مستقیمی بر کاهش زمان آموزش دارد.
- بهترین عملکرد زمانی با ترکیب دو هسته و fp16 به دست می‌آید، در حالی که برای حفظ دقت بهتر است از bf16 استفاده شود.

اگر همین کد ها را برای `mixed_precision="fp8"` اضافه کنیم، برای هر دو حالت یعنی یک ماشین و یک هسته و همچنین یک ماشین و دو هسته با ارور های زیر مواجه می شویم:

Traceback (most recent call last)

```
<File "/home/dmls/aqdam/tmrin4/3/Q3fp8.py", line 10, in <module>
accelerator = Accelerator(mixed_precision="fp8")
```

[illegible]

File "/home/shared_files/pytorch_venv/lib/python3.11/site-packages/accelerate/accelerator.py", line 425, in
__init

```
)self.state = AcceleratorState
```

^^ ^^ ^^ ^^ ^^ ^^ ^^ ^^ ^^ ^^ ^^ ^^ ^^ ^^ ^^ ^^

File "/home/shared_files/pytorch_venv/lib/python3.11/site-packages/accelerate/state.py", line 874, in __init

```
)raise ValueError
```

```
.ValueError: Using `fp8` precision requires `transformer_engine` or `MS-AMP` to be installed
```

```
:Traceback (most recent call last)
```

```
<File "/home/shared_files/pytorch_venv/bin/accelerate", line 8, in <module>
```

```
sys.exit(main())
```

^ ^ ^ ^ ^ ^

File "/home/shared_files/pytorch_venv/lib/python3.11/site-packages/accelerate/commands/accelerate_cli.py", line 48, in main

```
args.func(args)
```

File "/home/shared_files/pytorch_venv/lib/python3.11/site-packages/accelerate/commands/launch.py", line 1168,

in launch_command

```
simple_launcher(args)
```

File "/home/shared_files/pytorch_env/lib/python3.11/site-packages/accelerate/commands/launch.py", line 763, in simple_launcher

```
raise subprocess.CalledProcessError(returncode=process.returncode, cmd=cmd)
```

```
subprocess.CalledProcessError: Command '['/home/shared_files/pytorch_venv/bin/python', 'Q3fp8.py']' returned
non-zero exit status 1
```

```
:Traceback (most recent call last)
```

```
<File "/home/dmls/aqdam/tmrin4/3/Q3fp8.py", line 10, in <module
```

```
accelerator = Accelerator(mixed_precision="fp8")
```

[illegible]

File "/home/shared_files/pytorch_venv/lib/python3.11/site-packages/accelerate/accelerator.py", line 425, in
__init

```
)self.state = AcceleratorState
```

^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^

File "/home/shared_files/pytorch_venv/lib/python3.11/site-packages/accelerate/state.py", line 874, in __init

```
)raise ValueError
```

```
.ValueError: Using `fp8` precision requires `transformer_engine` or `MS-AMP` to be installed
```

```
:Traceback (most recent call last)
```

```
<File "/home/shared_files/pytorch_venv/bin/accelerate", line 8, in <module>
```

```
sys.exit(main())
```

^ ^ ^ ^ ^ ^

File "/home/shared_files/pytorch_venv/lib/python3.11/site-packages/accelerate/commands/accelerate_cli.py", line 48, in main

```
args.func(args)
```

File "/home/shared_files/pytorch_venv/lib/python3.11/site-packages/accelerate/commands/launch.py", line 1168,

in launch_command

```
simple_launcher(args)
```

File "/home/shared_files/pytorch_venv/lib/python3.11/site-packages/accelerate/commands/launch.py", line 763, in simple_launcher

```
raise subprocess.CalledProcessError(returncode=process.returncode, cmd=cmd)
```

```
subprocess.CalledProcessError: Command '['/home/shared_files/pytorch_venv/bin/python', 'Q3fp8.py']' returned
```

```

1
.non-zero exit status 1

```

که به این دلیل است که :

استفاده از دقت fp8 نیازمند نصب کتابخانه transformer_engine یا MS-AMP است که به طور پیش فرض در محیط نصب نشده‌اند و باید نصب شوند.

کتابخانه transformer_engine: این کتابخانه به طور خاص برای تسريع مدل‌های ترنسفورمر در دقت fp8 طراحی شده است.

کتابخانه MS-AMP: این کتابخانه برای استفاده از دقت fp8 در پردازش مدل‌های مختلف است.

pip install ms-amp و pip install transformer-engine

با اجرای این دستور ها در کلاستر با این مشکل مواجه می شویم:

WARNING: Certificate did not match expected hostname

که ما اجازه ی نصب در کلاستر را نداریم :

علت اصلی این خطا چیست؟

- عدم نصب کتابخانه‌های لازم برای fp8:

دقت fp8 به طور پیش فرض توسط کتابخانه accelerate یا PyTorch پشتیبانی نمی‌شود و نیازمند نصب کتابخانه های ذکر شده است.

- ویژگی جدید و خاص بودن fp8:

دقت FP8 فناوری جدیدی است که برای کارت‌های گرافیکی پیشرفته (مثل NVIDIA Hopper) طراحی شده تا با حافظه کمتر و سرعت بالاتر پردازش کند. این دقت هنوز به طور گسترده پشتیبانی نمی‌شود.

- عدم سازگاری سخت‌افزار:

حتی اگر این کتابخانه‌ها نصب باشند، اگر سخت‌افزار (مثل پردازنده گرافیکی) از FP8 پشتیبانی نکند، این ویژگی کار نخواهد کرد.

۴:

الف:

```
model = FeedforwardNeuralNetModel(input_dim, hidden_dim, output_dim)
torch.save(model.state_dict(), "model.pth")
```

یک شیء از مدل شبکه عصبی به نام FeedforwardNeuralNetModel با input_dim، hidden_dim و output_dim ایجاد می‌شود.

وزن‌ها و پارامترهای مدل با استفاده از state_dict ذخیره شده و در فایل "model.pth" سیو می‌شود.

```
model.load_state_dict(torch.load("model.pth"))
model = model.to("cpu")
model.eval()
```

پارامترهای ذخیره‌شده مدل بارگذاری می‌شود.

مدل به حالت CPU منتقل می‌شود و در حالت eval() قرار می‌گیرد، که در این حالت لایه‌هایی مثل Dropout و BatchNorm رفتار متفاوتی نسبت به حالت آموزش دارند.

```
with profile(
    activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
```

```

profile_memory=True,
record_shapes=True
) as prof:
    for i, (inputs, _) in enumerate(test_loader):
        if i >= 100:
            break
        model(inputs)

```

- هر دو فعالیت‌های CPU و CUDA بررسی می‌شوند تا بتوان عملکرد مدل را روی هر دو سخت‌افزار سنجید.
- با فعال‌سازی گزینه `profile_memory=True`، میزان حافظه مصرفی توسط مدل ثبت می‌شود.
- با `record_shapes=True` ابعاد ورودی و خروجی لایه‌ها در حین اجرا بررسی می‌شود.
- مدل برای حداکثر ۱۰۰ داده از مجموعه آزمایشی (`test_loader`) اجرا می‌شود. این محدودیت باعث می‌شود تحلیل سریع‌تر انجام شود و نمونه‌ای از عملکرد مدل بررسی گردد.

```

results = [
    [m.key, f"{m.cpu_time_total:.2f} ms", f"{m.self_cpu_memory_usage /
1024:.2f} KB"]
    for m in prof.key_averages() if any(x in m.key.lower() for x in
["relu", "batch_norm", "linear"])
]

```

- داده‌های پروفایل فیلتر می‌شود تا فقط لایه‌های ReLU، BatchNorm و Linear بررسی شوند.
- برای هر لایه، کل زمان پردازش در CPU (`cpu_time_total`) و میزان حافظه مصرفی (`self_cpu_memory_usage`) استخراج می‌شود.

```

print(tabulate(results, headers=["Module", "CPU Time (ms)", "Memory
(KB)"], tablefmt="grid"))

```

- داده‌ها با استفاده از تابع `tabulate` به شکل جدول نمایش داده می‌شوند.
- ستون‌های جدول شامل نام ماژول، زمان پردازش در CPU (میلی ثانیه) و مصرف حافظه (کیلوبایت) است.

نتیجه:

Module	CPU Time (ms)	Memory (KB)
aten::linear	63078.30 ms	0.00 KB
aten::batch_norm	19445.90 ms	0.00 KB
aten::_batch_norm_impl_index	18515.63 ms	0.00 KB
aten::native_batch_norm	15918.76 ms	-150.00 KB
aten::relu	11459.66 ms	0.00 KB

۱. لایه Linear:

بیشترین زمان پردازش را دارد. علت این موضوع، پیچیدگی محاسبات ماتریسی در لایه‌های خطی است که محاسبات زیادی انجام می‌دهند. لایه Linear بیشترین بار پردازشی را دارد و می‌تواند گلوگاه عملکرد مدل باشد. مصرف حافظه آن ۰.۰۰ KB گزارش شده است. این مقدار می‌تواند به دلیل بهینه‌سازی حافظه توسط PyTorch یا استفاده مشترک از حافظه باشد. با این حال، به طور معمول لایه‌های خطی به دلیل انجام ضرب ماتریسی سنگین، مصرف حافظه زیادی دارند.

۲. لایه BatchNorm:

زمان پردازش متوسط دارد. این لایه برای نرمال‌سازی داده‌ها استفاده می‌شود که شامل محاسباتی مانند میانگین و واریانس است. لایه BatchNorm نیز زمان قابل توجهی می‌گیرد اما بهینه‌تر از Linear است. برای aten::batch_norm مصرف حافظه ۰.۰۰ KB و برای aten::native_batch_norm مقدار -۱۵۰.۰۰ KB گزارش شده است. مقدار منفی نشان‌دهنده آزادسازی حافظه در حین اجرا است. BatchNorm معمولاً به حافظه برای ذخیره میانگین و واریانس نیاز دارد، اما این مقدار بعد از استفاده آزاد می‌شود.

۳. لایه ReLU:

کمترین زمان پردازش را دارد. عملکرد آن بسیار ساده است و فقط مقادیر منفی را به صفر تبدیل می‌کند، به همین دلیل زمان بسیار کمی صرف می‌کند. لایه ReLU از نظر پردازشی بسیار سریع و سبک است.

مصرف حافظه ۰.۰۰ KB دارد. این مقدار منطقی است چون ReLU فقط مقادیر منفی را صفر می‌کند و نیاز به حافظه اضافی ندارد.

ب:

```
import torch
from torch.profiler import profile, ProfilerActivity
```

```

import numpy as np
from torch.utils.data import TensorDataset, DataLoader
import torch.nn as nn
from tabulate import tabulate

test_x = np.load('test_x.npy').astype(np.float32)
test_y = np.load('test_y.npy').astype(np.float32)
test_dataset = TensorDataset(torch.tensor(test_x).float(),
                              torch.tensor(test_y).long())

train_x = np.load("train_x.npy").astype(np.float32)
train_y = np.load("train_y.npy").astype(np.float32)
train_dataset = TensorDataset(torch.tensor(train_x).float(),
                              torch.tensor(train_y).long())

batch_size = 100
n_iters = 3000
num_epochs = n_iters / (len(train_dataset) / batch_size)
num_epochs = int(num_epochs)

train_loader = torch.utils.data.DataLoader(dataset=train_dataset,
                                           batch_size=batch_size,
                                           shuffle=True)

test_loader = torch.utils.data.DataLoader(dataset=test_dataset,
                                           batch_size=batch_size,
                                           shuffle=False)

class FeedforwardNeuralNetModel(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim, activation_fn):
        super(FeedforwardNeuralNetModel, self).__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.bn1 = nn.BatchNorm1d(hidden_dim)
        self.act1 = activation_fn()
        self.fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.bn2 = nn.BatchNorm1d(hidden_dim)
        self.act2 = activation_fn()
        self.fc3 = nn.Linear(hidden_dim, hidden_dim)
        self.bn3 = nn.BatchNorm1d(hidden_dim)
        self.act3 = activation_fn()
        self.fc4 = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        out = self.fc1(x)
        out = self.bn1(out)

```

```

        out = self.act1(out)
        out = self.fc2(out)
        out = self.bn2(out)
        out = self.act2(out)
        out = self.fc3(out)
        out = self.bn3(out)
        out = self.act3(out)
        out = self.fc4(out)
        return out

input_dim = test_x.shape[1]
hidden_dim = 64
output_dim = len(torch.unique(torch.tensor(test_y)))

activations = {
    "ReLU": nn.ReLU,
    "Tanh": nn.Tanh,
    "Sigmoid": nn.Sigmoid,
    "GeLU": nn.GELU
}

for name, activation_fn in activations.items():
    print(f"\nRunning for activation function: {name}")
    model = FeedforwardNeuralNetModel(input_dim, hidden_dim, output_dim,
    activation_fn)
    model.eval()

    with profile(
        activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
        profile_memory=True,
        record_shapes=True
    ) as prof:
        with torch.no_grad():
            for i, (inputs, _) in enumerate(test_loader):
                if i >= 100:
                    break
                outputs = model(inputs)

    print(f"Profiling results for {name}:")
    print(prof.key_averages().table(sort_by="cpu_time_total",
row_limit=10))
    print("\n")

```

برای هر تابع فعال‌سازی، یک مدل جدید ساخته می‌شود و با استفاده از ابزار Profiler، زمان پردازش و مصرف حافظه مدل هنگام پیش‌بینی داده‌های تست (تا ۱۰۰ نمونه) تحلیل می‌شود. نتایج به‌دست‌آمده برای هر تابع فعال‌سازی در فایلی ذخیره می‌شود تا مشخص شود کدام تابع فعال‌سازی از نظر زمان اجرا و مصرف حافظه عملکرد بهتری دارد.

Name	Self CPU %	Self CPU	CPU total %	CPU total	CPU time avg	CPU Mem	Self CPU Mem	# of Calls
aten::linear	0.27%	2.009ms	68.26%	499.892ms	1.250ms	8.09 Mb	0 b	400
aten::addmm	66.82%	489.307ms	67.45%	493.899ms	1.235ms	8.09 Mb	8.09 Mb	400
enumerate(DataLoader)#_SingleProcessDataLoaderIter._...	10.91%	79.897ms	26.74%	195.789ms	1.939ms	19.61 Mb	-20.36 Kb	101
aten::select	7.07%	51.781ms	8.99%	65.869ms	3.293us	0 b	0 b	20000
aten::stack	1.88%	13.750ms	6.80%	49.772ms	248.860us	19.61 Mb	0 b	200
aten::batch_norm	0.14%	1.013ms	4.61%	29.340ms	97.800us	7.32 Mb	0 b	300
aten::_batch_norm_impl_index	0.31%	2.277ms	3.87%	28.327ms	94.423us	7.32 Mb	0 b	300
aten::native_batch_norm	3.05%	22.368ms	3.46%	25.328ms	84.427us	7.32 Mb	-150.00 Kb	300
aten::as_strided	3.05%	22.347ms	3.05%	22.347ms	0.721us	0 b	0 b	31000
aten::unsqueeze	1.94%	14.207ms	2.88%	21.096ms	2.110us	0 b	0 b	10000

Self CPU time total: 732.293ms

Running for activation function: Tanh									
Profiling results for Tanh:									
	Name	Self CPU %	Self CPU	CPU total %	CPU total	CPU time avg	CPU Mem	Self CPU Mem	# of Calls
	aten::linear	0.26%	1.874ms	69.09%	496.286ms	1.241ms	8.09 Mb	0 b	400
	aten::addmm	67.71%	486.373ms	68.32%	490.776ms	1.227ms	8.09 Mb	8.09 Mb	400
enumerate(DataLoader)#_SingleProcessDataLoaderIter, ...	aten::select	10.87%	78.099ms	26.22%	188.346ms	1.865ms	19.61 Mb	-20.64 Kb	101
	aten::stack	7.03%	50.465ms	8.93%	64.167ms	3.208us	0 b	0 b	28000
	aten::as_strided	1.73%	12.460ms	6.38%	45.833ms	229.165us	19.61 Mb	0 b	200
	aten::unsqueeze	3.04%	21.811ms	3.04%	21.811ms	0.704us	0 b	0 b	31000
	aten::batch_norm	2.00%	14.343ms	2.94%	21.152ms	2.115us	0 b	0 b	10000
	aten::batch_norm	0.14%	1.019ms	2.43%	17.434ms	58.113us	7.32 Mb	0 b	300
	aten::_batch_norm_impl_index	0.32%	2.267ms	2.29%	16.415ms	54.717us	7.32 Mb	0 b	300
	aten::tanh	2.14%	15.339ms	2.14%	15.339ms	51.130us	7.32 Mb	7.32 Mb	300
Self CPU time total: 718.326ms									

```

Running for activation function: Sigmoid
Profiling results for Sigmoid:

```

Name	Self CPU %	Self CPU	CPU total %	CPU total	CPU time avg	CPU Mem	Self CPU Mem	# of Calls
aten::linear	0.24%	1.929ms	67.78%	551.120ms	1.378ms	8.09 Mb	0 b	400
aten::addmm	66.47%	540.536ms	67.06%	545.280ms	1.363ms	8.09 Mb	8.09 Mb	400
enumerate(DataLoader)#_SingleProcessDataLoaderIter._...	10.49%	85.306ms	25.61%	208.288ms	2.062ms	19.61 Mb	-21.49 Kb	101
aten::select	6.88%	55.963ms	8.74%	71.101ms	3.555us	0 b	0 b	20800
aten::stack	1.78%	14.499ms	6.35%	51.637ms	258.185us	19.61 Mb	0 b	200
aten::sigmoid	3.94%	32.009ms	3.94%	32.009ms	106.697us	7.32 Mb	7.32 Mb	300
aten::as_strided	2.96%	24.093ms	2.96%	24.093ms	0.777us	0 b	0 b	31000
aten::unsqueeze	1.99%	16.165ms	2.92%	23.775ms	2.377us	0 b	0 b	10000
aten::batch_norm	0.14%	1.122ms	2.56%	20.806ms	69.353us	7.32 Mb	0 b	300
aten::_batch_norm_impl_index	0.30%	2.446ms	2.42%	19.684ms	65.613us	7.32 Mb	0 b	300

```

Self CPU time total: 813.154ms

```

Running for activation function: GeLU

Profiling results for GeLU:

	Name	Self CPU %	Self CPU	CPU total %	CPU total	CPU time avg	CPU Mem	Self CPU Mem	# of Calls
enumerate(DataLoader)# SingleProcessDataLoaderIter._...	aten::linear	0.34%	3.151ms	62.55%	573.920ms	1.435ms	8.09 Mb	0 b	400
	aten::addmm	60.86%	558.359ms	61.57%	564.980ms	1.412ms	8.09 Mb	8.09 Mb	400
	aten::select	11.46%	105.100ms	26.96%	247.395ms	2.449ms	19.61 Mb	-22.05 Kb	101
	aten::stack	6.98%	64.081ms	8.82%	80.950ms	4.048us	0 b	0 b	20900
	aten::batch_norm	1.99%	18.298ms	6.65%	60.980ms	304.900us	19.61 Mb	0 b	200
	aten::_batch_norm_impl_index	0.26%	2.366ms	5.88%	53.913ms	179.710us	7.32 Mb	0 b	300
	aten::native_batch_norm	4.43%	40.674ms	4.91%	45.034ms	150.113us	7.32 Mb	-150.00 Kb	300
	aten::gelu	4.48%	41.136ms	4.48%	41.136ms	137.120us	7.32 Mb	7.32 Mb	300
	aten::as_strided	2.92%	26.748ms	2.92%	26.748ms	0.863us	0 b	0 b	31000
	Self CPU time total: 917.503ms								

تابع ReLU بسیار سریع و کارآمد است. تنها یک عملیات ساده‌ی فیلتر کردن مقادیر منفی است (اگر ورودی منفی باشد خروجی صفر است). مصرف حافظه تقریباً ناچیز است، بنابراین ReLU برای مدل‌هایی که نیاز به سرعت دارند مناسب است.

Tanh عملکرد مشابهی با ReLU دارد اما با کمی سرعت بیشتر. خروجی آن بین -۱ و ۱ است که به مرکز داده‌ها کمک می‌کند. مصرف حافظه آن نیز مشابه ReLU است، اما محاسبات پیچیده‌تری نسبت به ReLU دارد.

تابع Sigmoid کندتر از ReLU و Tanh است. خروجی آن بین ۰ و ۱ است و برای مسائل باینری مناسب است. محاسبات نمایی باعث کندتر شدن این تابع نسبت به ReLU و Tanh شده است.

تابع GeLU پیچیده‌ترین تابع فعال‌سازی در این مقایسه است. به دلیل داشتن محاسبات پیچیده‌تر (شامل تابع خطای گاوسی)، زمان پردازش آن بیشتر از سایر توابع است. GeLU معمولاً برای شبکه‌های عمیق‌تر و مدل‌های پیشرفته مثل BERT استفاده می‌شود که به دقت بالاتری نیاز دارند. اگرچه زمان پردازش آن بیشتر است، اما در بهبود دقت مدل‌ها مؤثر است.

بنابراین: Tanh سریع‌ترین تابع فعال‌سازی است. GeLU کندترین تابع است و زمان پردازش آن تقریباً ۱.۲۵ برابر بیشتر از Tanh است. ReLU نیز عملکرد خوبی دارد و تفاوت زیادی با Tanh ندارد. Sigmoid از نظر زمان پردازش از ReLU و Tanh کندتر است. مصرف حافظه در همه توابع تقریباً ۸.۰۹ Mb است که نشان می‌دهد تفاوت زیادی از این نظر ندارند. مقدار حافظه Self Memory برای همه توابع ۰ است که احتمالاً به دلیل بهینه‌سازی حافظه در PyTorch است. از نظر پیچیدگی محاسباتی، ReLU و Tanh توابع ساده‌تری دارند و زمان پردازش کمتری دارند. Sigmoid به دلیل محاسبات نمایی کندتر است و GeLU به دلیل محاسبات پیچیده‌تر، کندترین تابع فعال‌سازی است.